

研讨题 9-1

ISA (Instruction Set Architecture)

包含如下内容：

- **processor state (e.x Y86)**

寄存器：%rax ~ %r14

条件码：ZF, OF, SF

程序计数器 PC

程序状态：指示异常状态

内存：字节对齐，小端序

- **指令**

addq, pushq, ret (**Set of str**)

如何将指令编码为字节 (**Dict [str, bytes]**)

Y86-64 ISA

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

一些规律：

- 第一字节为操作码，高四位表示指令码(icode)，低四位为功能码(ifun), fn或占位符0，其中fn表示opq或条件跳转和传送的更具体操作
- 第二字节中如果只涉及到一个寄存器则剩下一个位置用F填充
- 跳转地址，立即数，内存偏移都用8字节表示

- cmovXX

rrmovq

2	0
---	---

cmovle

2	1
---	---

cmovl

2	2
---	---

cmove

2	3
---	---

cmovne

2	4
---	---

cmovge

2	5
---	---

cmovg

2	6
---	---

fn = 0 时等价于 rrmovq

- OPq

addq

6	0
---	---

subq

6	1
---	---

andq

6	2
---	---

xorq

6	3
---	---

- jXX 与 cmovXX 同理

采用Y86中绝对地址进行跳转，简化CPU实现逻辑，契合教学重点

- 寄存器ID:

%rax	0	%r8	8
%rcx	1	%r9	9
%rdx	2	%r10	A
%rbx	3	%r11	B
%rsp	4	%r12	C
%rbp	5	%r13	D
%rsi	6	%r14	E
%rdi	7	No Register	F

指令转换

Y86: `irmovq $0xabcd, %rdx`
 encoding: 30 f2 cd ab 00 00 00 00 00 00

Y86: `popq %rsi`
 encoding: b0 6f

Y86: `andq %rax, %rbx`
 encoding: 62 03

X86 vs Y86

```
int fix_ele(fix_matrix a, size_t i, size_t j) {
    return a[i][j]; /*size_A 16 * 16 */
}
```

X86版

```
salq $6, %rsi
addq %rsi, %rdi
movl (%rdi,%rdx,4), %eax
ret
```

Y86版:

```
addq %rsi, %rsi
addq %rsi, %rsi
addq %rsi, %rsi
```

```
addq %rsi, %rsi
addq %rsi, %rsi
addq %rsi, %rsi
addq %rdx, %rdx
addq %rdx, %rdx
addq %rsi, %rdx
addq %rdx, %rdi
irmovq $0, %rax
mrmovq (%rdi), %rax
```

RISC VS CISC

- CISC: 用一条功能强大的指令完成一个复杂的任务, 代码密度高
- RISC: 用多条简单的指令组合来完成复杂任务, 依赖编译器优化, 硬件简单, 能效比高, 设计灵活

总结

如今CISC和RISC的界限已经变得模糊, 现代CISC处理器会先将复杂的 x86 指令解码成类似 RISC 的、更简单的微操作, 然后再交由一个高效的、基于流水线的 RISC-like 核心去执行; 现代 RISC 处理器 (如 ARM) 也在不断添加更复杂的专用指令, 以提高特定任务的处理效率。